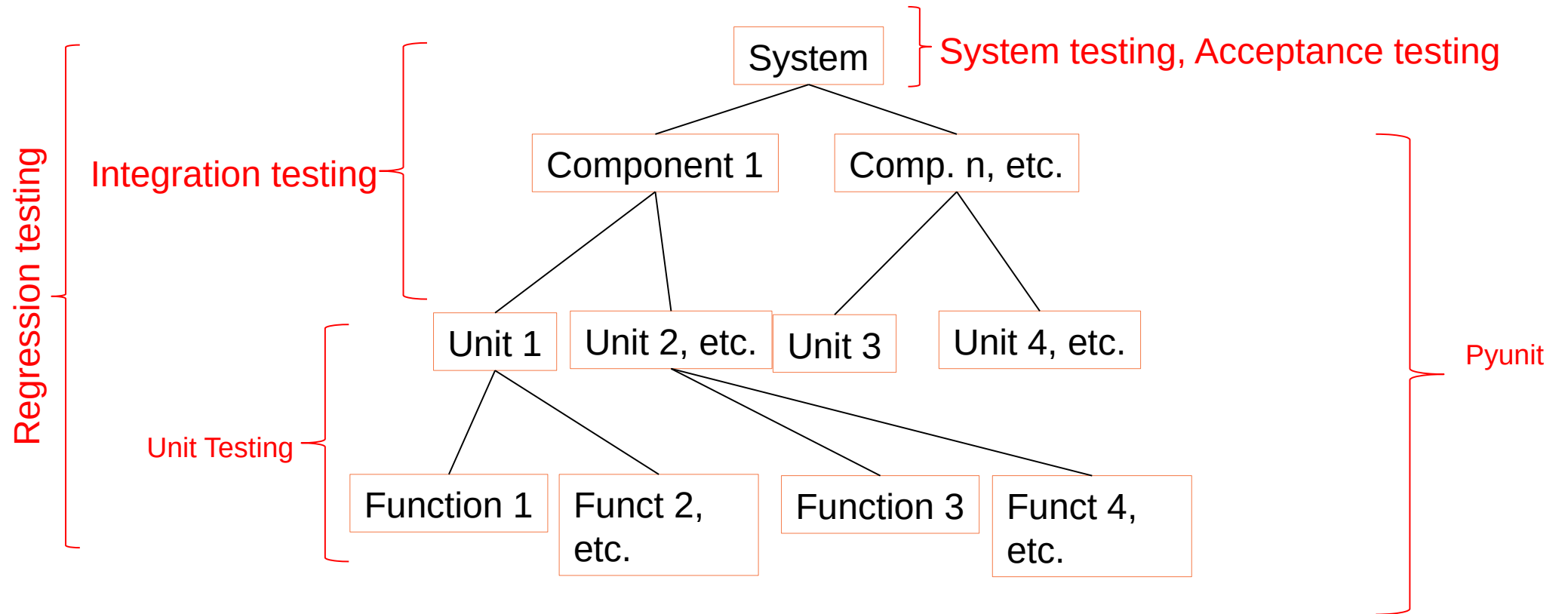

Test Levels

Recall: Software System Decomposition

Software System Decomposition



Test Driven Development (TDD)

- A philosophy whereby we start by asking the question “How can I test this?” or “Can this be tested?” or “How will I test this?”
- Not uniquely tied to any particular testing phase – but it a way of thinking about any type of testing / development.
- Why bother with TDD?
- It places an emphasis on thinking about the operation of the software once completed.
- If it cannot be tested, it should not be implemented?
- Or: no point implementing something if we cannot test it?
- Effective thinking for the early identification of possible defects.
- Forces us to clarify and improvement the completeness of our thinking earlier – which is typically a desirable economic proposition.

Component testing

A component can be a single unit or a collection of units

The first dynamic test stage is sometimes considered to be component testing – though in a contemporary sense, this is probably framework driven unit testing.

Performed on components in **isolation**.

Stubs, **drivers** and **simulators** may be used.

May include testing of **some non-functional** characteristics.

Typically, a technical activity, performed by **programmers**.

Integration testing

Many components may be *integrated* into a large system.

When components are integrated into a system, we may need to perform dedicated integration testing.

Integration testing is based on the build and integration sequence

An incremental approach may be used – testing for high level features first.

A technical activity, performed by the developers, like component testing. Sometimes performed by testers?

Software integration and Component Integration testing

Integration and integration testing is not well understood

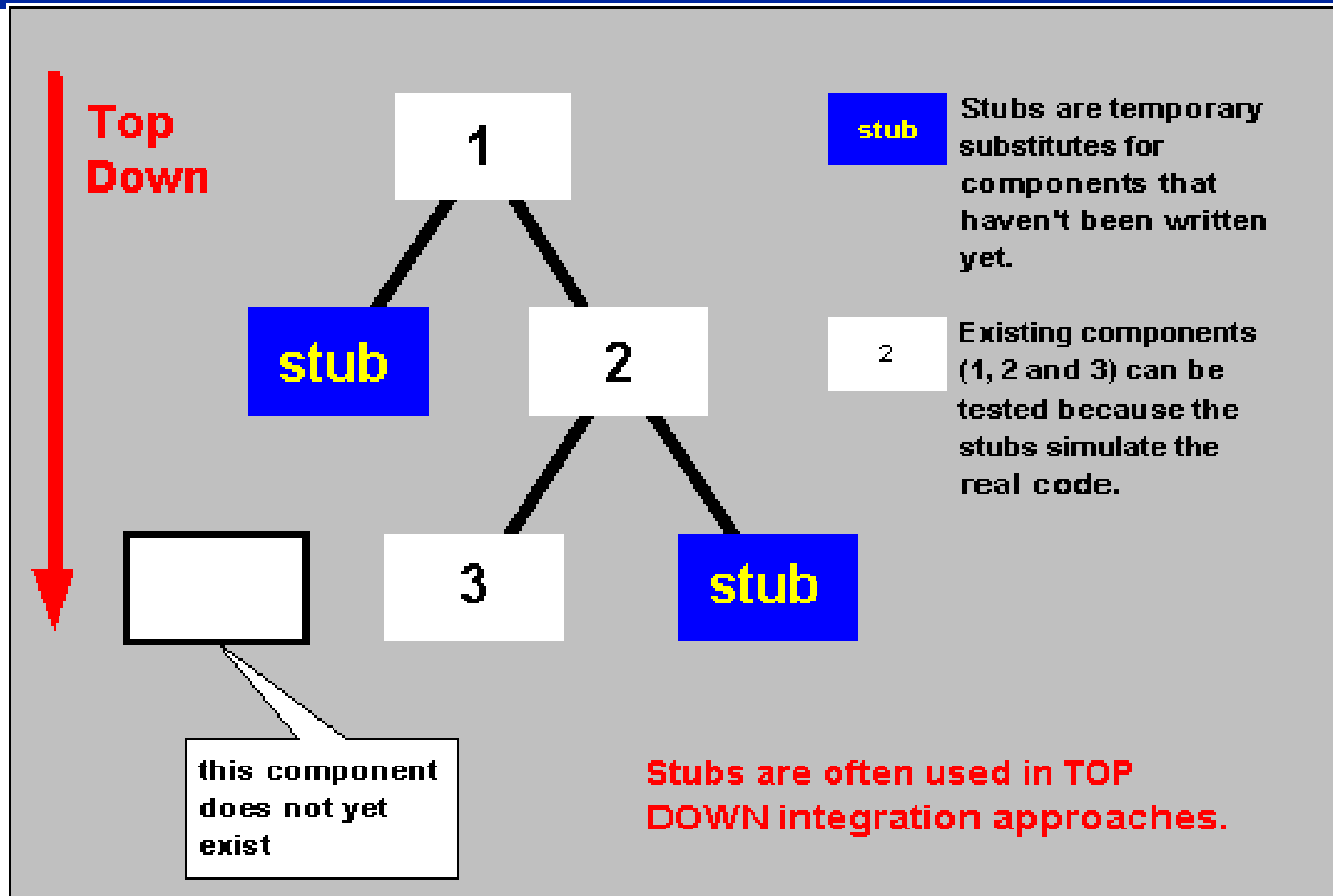
In the coding stage, you are performing "integration in the very small"

Strategies for coding and integration:

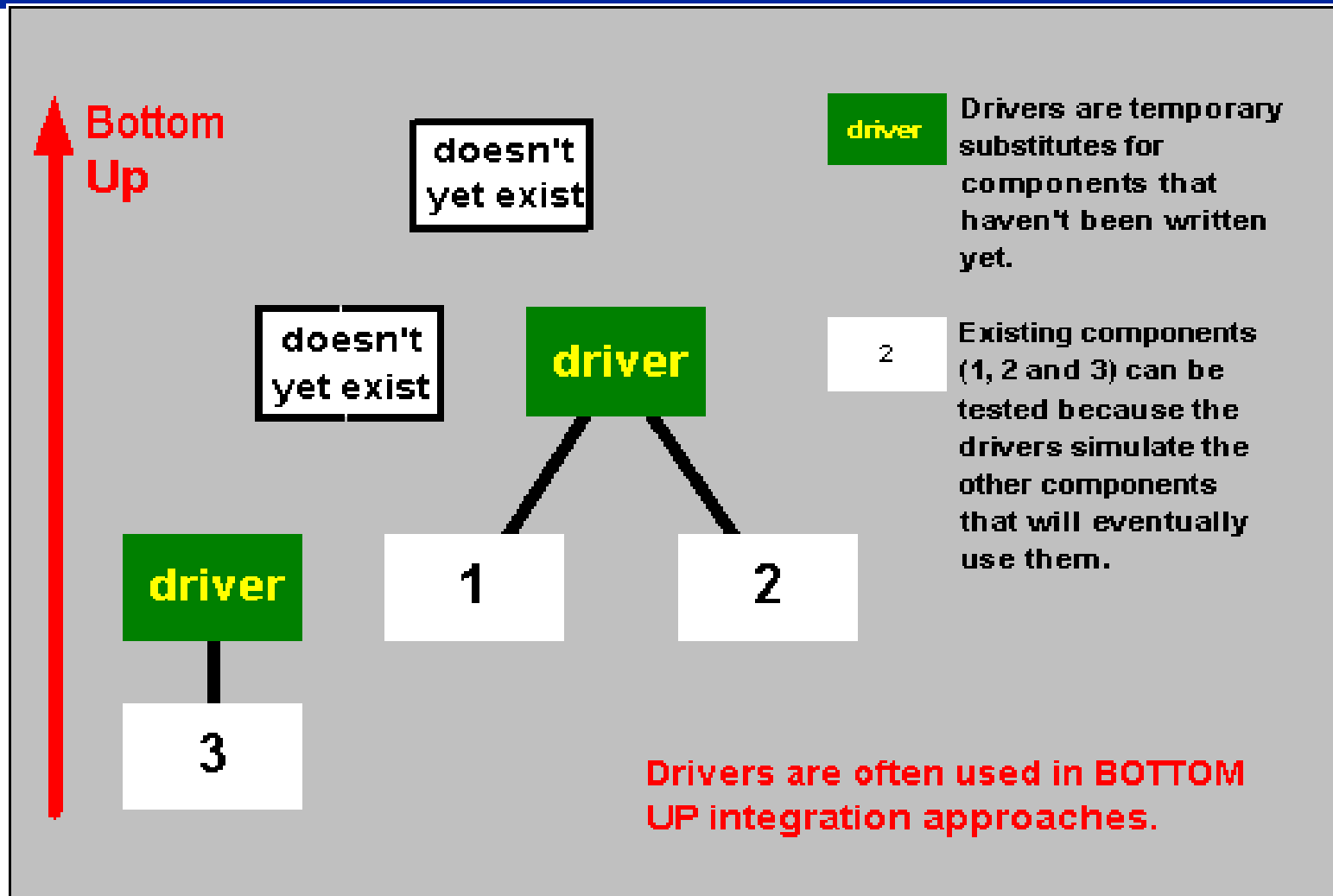
bottom up, top down, "big bang"

appropriate in different situations

Stubs and top down testing



Drivers and bottom up testing



What does a stub or driver look like?

Stubs and drivers can take many forms.

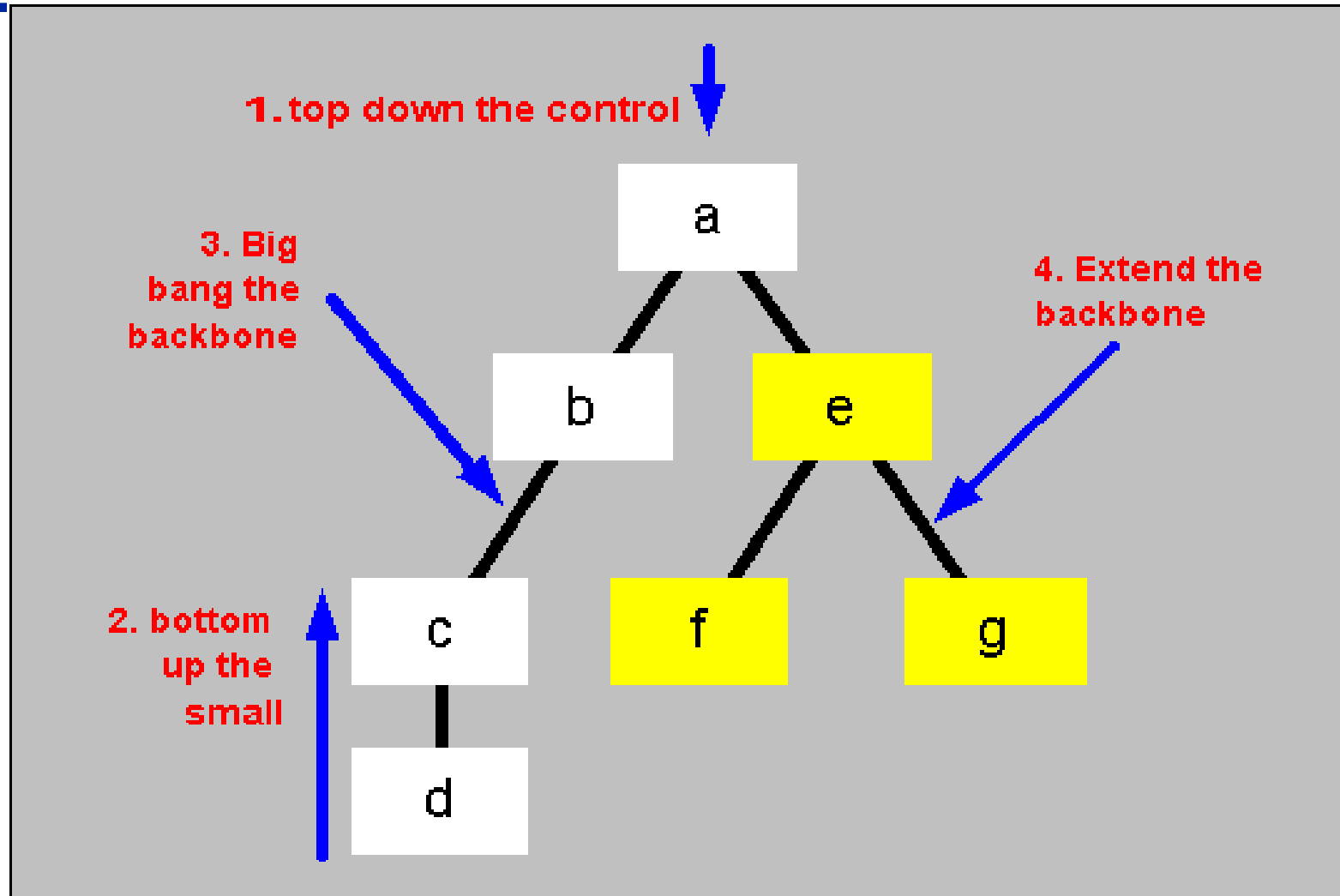
1. They can be implemented in a script that interacts with the software under test.
2. They can be implemented as a separate program, perhaps written in the same programming language (perhaps not).
3. May use a 3rd party product to build a stub or driver.

Stubs/Drivers: May include logging and recording for tests, and automated checking of logs for errors.

Note of caution on terminology

The distinction that we have drawn between Stubs and Drivers may not be generally held in practice.

Mixed integration strategy



Component Integration testing

Tests **designed to explore direct and indirect interfaces** and consistency between components.

First stage where products of several programmers are combined

Can **reveal** consequences of **inconsistent assumptions or bad communication**

Usually performed by an individual within a development group – but can be the responsibility of a small dedicated team.

Definition of interfaces

Statements which transfer control between programs

Parameters passed from program to program.

Variables defined at the time of transfer

Interface testing / defects

White-box oriented

Integration faults reveal project communication failures

Transfer of control to the wrong routine

Wrong type (hierarchical or lateral) of transfer of control

Incorrect type, number or order of parameters.

Programs validate common data inconsistently

Read-only parameters or global data that is written to.

Inter-module parameter checking

Does the called routine explicitly check input parameters?

Does the calling routine check

called routine return status?

returned values?

Programming or interface standards should define whether callers, called or both routines perform checking and under what circumstances.

System and Acceptance Testing

System and acceptance testing **focus on the testing of complete systems**

System testing has multiple "drivers".

System testing

A **systematic demonstration that all features are available and work as specified**

Coverage of all documented features (not code), conditions, system states is the imperative

Tests designed around baseline document(s) [i.e. not based on the implementation details]

Functional and non-functional requirements are covered.

Often, non-functional requirements can be poorly defined and are last to be performed – but can be difficult to resolve issues in this area (e.g. functionality is delivered but performance is poor).

Non-functional requirements are therefore just as important as functional requirements but may be last to be evaluated (and indeed, sometimes unclear until the operational environment is made available).

Functional and non-functional system testing

System functional and non-functional testing

Functional system testing

- concerned with **functional requirements**

- fault-detection a major objective

- "**what** the system must do"

Non-functional system testing

- concerned with **non-functional** or technical requirements

- "**how** the system does what it does".

Acceptance testing (UAT)

Done from an end user's viewpoint

Fit with business process is the imperative: does it help the users do their job?

May place an emphasis on essential features: some delivered features may not be completely tested.

Testing based around how users use the system.

Usual to assume that all major faults have been removed and the system works (be careful with assumptions...!).

Acceptance tests:

- usually a smaller-scale test than system test

- can be a selected sub-set of system tests (perhaps in a different environment)

Acceptance may not require its own test

- Sometimes based on satisfactory conduct of system tests, often witnessed.

- Depends on the contractual position

Q. Do google run acceptance tests?

Requirements-based testing

Acceptance tests and some system tests are usually requirements-based

We can scan requirements documents:

- which features should be provided
- what conditions should be covered

Requirements based tests:

- can help to demonstrate that the supplier has met every requirement
- less influenced by the solution provided by the supplier.

Problems with requirements

Requirements don't usually give us enough information to test
intents, not detailed implementation
need to identify features to test
many details might be assumed to exist, but can't be identified from
requirements
e.g. field validation, navigation paths, data integrity rules all added by
developers.

Alpha and beta testing

Often used by suppliers of packages (particularly shrink-wrapped)

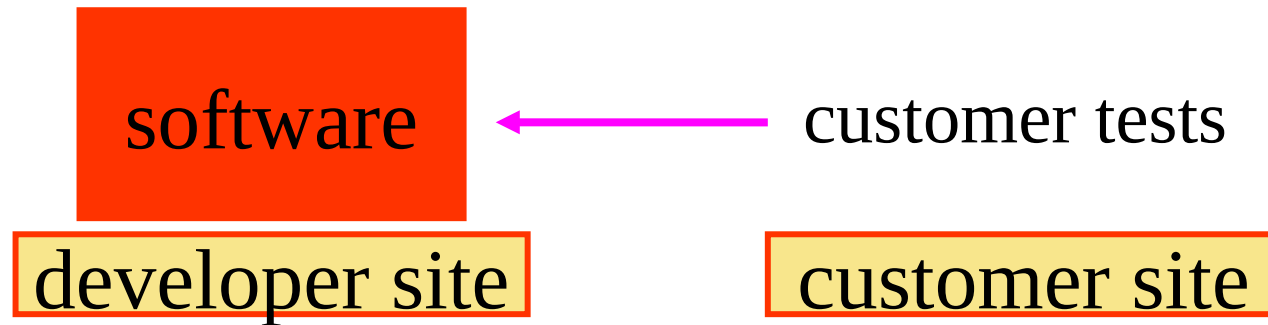
Where supplier wishes to receive feedback from actual or potential customers

Alpha testing normally takes place on the supplier site

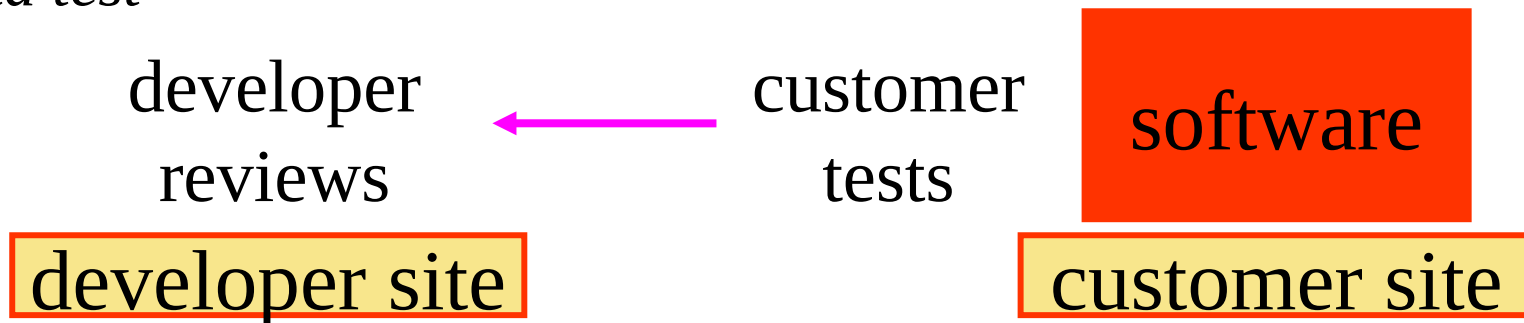
Beta testing usually conducted by users on their site.

Alpha & Beta test

Alpha test



Beta test



Alpha and beta testing - intent

Assess reaction of marketplace to the product

Are major features missing?

Do new features 'miss the point'?

Is product ready for release?

Some suppliers leave faults in the software to get bug reports returned to gauge?

where software is being used most

where users are most sensitive to faults.